

SVF

Project Documentation

What has been built, when, and why — from the first commit to the current state.

Field	Value
Last updated	2026-07-30
Repository	subha-sh-11/SVF (deployed on Vercel)
Contributor	subha-sh-11 (subhashraj_k@lorvenaistudio.com)
History span	2026-07-16 -> 2026-07-29 (19 commits)

1. What SVF Is

SVF is an internal web platform for a film-distribution operation covering the Nizam area theatre circuit. It has three distinct faces sharing one Next.js codebase and one PostgreSQL database:

- **Admin console** - manage theatres, representatives, roles/permissions, rate cards, and users.
- **Representative (Rep) app** - a guided mobile-style flow for theatre reps to submit daily show/sales reports.
- **Movies workspace** - upload an Excel daily-collections report and view/edit it as a live, faithful spreadsheet (Univer), with sharing and collaboration.

The recent and ongoing engineering focus (late July) has been making the Movies workspace render uploaded Excel files as an exact visual replica - colors, gridlines, merged cells, and column layout.

2. Technology Stack

Layer	Technology
Framework	Next.js ^15.1 (App Router)
UI	React 19, TypeScript 5, Tailwind CSS 3.4
Database	PostgreSQL via pg (SSL, pooled)
Spreadsheet engine	@univerjs/presets ^0.25 (canvas spreadsheet)
Excel I/O	exceljs ^4.4 (parse + write), xlsx ^0.18
Export	jspdf ^4.2 + jspdf-autotable ^5 (PDF), exceljs (Excel)
Rep SPA routing	react-router-dom ^6.30 (embedded SPA)
Legacy grid	jspreadsheet-ce ^5, jsuites ^6 (early experiments)
Hosting	Vercel (GitHub-linked, auto-deploy from main)

Auth is custom (no library): HMAC-SHA256-signed session tokens in cookies, bcrypt password hashing for reps.

3. High-Level Architecture

frontend/ - src/app/	Next.js app (the whole product) App Router
--------------------------	---

- login/	Admin + user login
- admin/	Admin console (dashboard, theatres, reps, roles, users)
- movies/	Movies list + /movies/[id] spreadsheet
- rep/[...slug]/	Catch-all mount for the Rep SPA
- api/	Route handlers (auth, movies, users, reports, ...)
- src/rep/	Representative SPA (React Router: pages, components, context)
- src/components/	MovieSheet, UniverSheet, admin widgets, RateCard, Sidebar
- src/lib/	auth, db, movies, roles, users, xlsxToUniver, univerToExcel, ...
- src/data/	theatres.json (seed data)
'- scripts/seed.mjs	Seeds the theatres table from theatres.json

Three parallel session types (each its own signed cookie):

Session	Cookie	Who	Access
Admin	svf_session	admin@svf.in	Everything; sees all movies
User	user_session	movies users	Own + shared movies
Rep	rep_session	approved reps	Rep app only
Impersonation	svf_impersonate	admin acting as a user	'Login as' a user

4. Chronological Timeline

Dates and messages are the actual git commits (IST).

Phase 0 - Foundations (16 Jul 2026)

Time	Commit	What landed
13:11	5cc0127	Initial commit - SVF admin frontend scaffold
13:14	434d4d5	Add frontend app - Next.js project, config
17:47	bcff6ed	DB-backed theatres + admin login - Postgres theatres table, seed.mjs, admin login, roles/reps/users libs, admin pages, CreateRoleModal, TheatreExplorer, CoverageCard

The theatre catalogue (districts -> centres -> theatres -> screens, with capacity and HFC totals) is seeded from src/data/theatres.json into Postgres.

Phase 1 - V1 Platform & Rep App (21 Jul 2026)

Time	Commit	What landed
17:20	02bf7d2	V1 - largest early drop (~9,100 lines): the Representative SPA (src/rep/*) with full page flow, the reports API, rate cards, auth verify/logout, me/my-theatres endpoints, movies scaffolding, Tailwind config, and the /rep/[...slug] mount
17:28	491d1d3	Skip ESLint during build (next.config) - Vercel deploy fix
17:47	811c69b	Skip ESLint during build (eslint.config) - Vercel deploy fix
17:50	ec92f11	Skip ESLint during build (vercel.json) - Vercel deploy fix

The three 'Skip ESLint' commits unblocked Vercel production builds.

Phase 2 - Univer Spreadsheets, Accounts & Sharing (22-24 Jul 2026)

When	Commit	What landed
Jul 22 18:02	8f6b548	Univer spreadsheets, app-user accounts, sharing, and fixes - second-largest drop

		(~7,100 lines): the Univer integration (UniverSheet.tsx, MovieSheet.tsx), xlsxToUniver.ts (Excel->Univer converter), movies DB APIs, admin users page, user login, whoami, movie_shares sharing
Jul 23 11:38	b911dc2	Fix movie-open crash + collab blank + loading card
Jul 23 16:22	d5fe800	ver2 - users admin, 'Login as' impersonation, univerToExcel.ts (Excel export), movies page overhaul, currentEmail impersonation support
Jul 23 16:32	b6b7fa1	ver2 - movies page refinements
Jul 23 16:45	b30fe46	ver2 - movies API + page + lib refinements
Jul 24 13:36	4da7022	ver2 - MovieSheet/UniverSheet toolbar & feature work, export tweaks

This phase turned 'movies' into a real collaborative spreadsheet product: upload -> convert -> render
-> auto-save -> share -> export.

Phase 3 - Spreadsheet Fidelity & Issue Fixing (27-29 Jul 2026)

When	Commit	What landed
Jul 27 13:53	15614fc	Issues Fixed - large MovieSheet + UniverSheet feature/rendering work (~626 lines)
Jul 27 16:52	15c7cb6	Issues Fixed - sanitize/rendering refinements
Jul 27 17:31	c5f63aa	Issues Fixed - converter/sanitize tweaks
Jul 27 18:06	bb9a090	Issues Fixed - UniverSheet toolbar additions, converter tweaks
Jul 29 12:28	4dbe7b8	issue fixed - xlsxToUniver + MovieSheet rendering fidelity
Jul 29 17:46	d442611	issue fixed - major xlsxToUniver (196-line churn) + MovieSheet fidelity pass; source Excel files committed for reference

Phase 4 - In Progress (30 Jul 2026, uncommitted)

Fidelity investigation into green audience columns and yellow gross columns (see Section 8). No commits yet.

5. Feature Detail by Subsystem

5.1 Admin Console (/admin/*)

- **Login** - admin@svf.in / Admin@321 (env-overridable), HMAC-signed svf_session cookie, 7-day expiry.
- **Dashboard** - analytics / coverage overview.
- **Theatres** - browse the Nizam theatre tree (district -> centre -> theatre -> screens) backed by the Postgres theatres table; capacity and HFC totals per theatre.
- **Representatives** - approve/reject/edit reps; assign reps to theatres.
- **Roles & permissions** - role editor with a fixed permission set: view_dashboard, view_theatres, edit_rates, assign_reps, manage_reps, manage_roles, export_data. Default roles: Super Admin, Manager, Representative. (Roles/users currently persist in localStorage, not the DB.)
- **Rate cards** - rate slabs / full-house values (RateCard.tsx, ratecard.ts).
- **Users** - manage movies-app users; 'Login as' impersonation to see the app exactly as that user.

5.2 Representative App (/rep/*)

A self-contained React Router SPA mounted via a Next catch-all route.

- **Auth** - email + password validated against an approved rep in the DB (reps table); bcrypt password hashes (rep-auth.ts); separate rep_session cookie; session restored on refresh via /api/me.
- **Flow** - Login -> Theatres (assigned only) -> Theatre Details -> Show Selection -> Sales Entry -> Report Review -> Success, plus a History page.
- **computeReport** (AppContext.jsx) - pure calculation from a seat plan + seats-sold map: per-category sold/unsold, max vs actual collection, difference, and occupancy.
- **Persistence** - submitted reports upsert into the reports table (/api/reports), keyed by (rep_id, key) so re-submitting a show updates it.

5.3 Movies Workspace (/movies) - the core feature

- **Upload** - an .xlsx daily-collections report is parsed with ExcelJS and converted to a Univer workbook snapshot by xlsxToUniver.ts.
- **Faithful rendering** - the converter preserves values, formulas, fills, fonts, borders, alignment, number formats, merged cells, column widths, row heights, and sheet tab colors. It reads each file's real theme palette from xl/theme/theme1.xml plus a legacy indexed-color fallback, so colors match the source exactly.
- **Storage** - three tables: movies (metadata), movie_sheets (the JSONB snapshot + version), movie_shares (access grants).
- **Collaboration** - clients auto-save (debounced) via PUT /api/movies/[id]/sheet; version increments on each save (last-write-wins); other clients poll ?meta=1 for a higher version and pull updates.
- **Sharing & access** - accessRole() resolves editor / viewer / none: admin and owner are always editors; others get their movie_shares role. Admin sees every movie; a user sees only owned or shared ones.
- **Export** - back to Excel (univerToExcel.ts) and PDF (jspdf + autotable).
- **Toolbar / editing** - spreadsheet toolbar features accumulated across the Phase 2-3 commits.

5.4 Auth Model

- HMAC-SHA256 tokens: base64url(email|exp).signature, verified with timingSafeEqual, 7-day max age.
- currentEmail() unifies admin + user sessions and honors impersonation (svf_impersonate) so movie access behaves as the impersonated user, while admin-only endpoints still check svf_session directly.
- Reps use a parallel scheme (rep_session, bcrypt hashes).

6. Data Model (PostgreSQL)

Tables are created lazily on first use (CREATE TABLE IF NOT EXISTS) and seeded/migrated in code.

Table	Purpose	Key columns
-------	---------	-------------

theatres	Nizam theatre catalogue (seeded)	id, district, centre, theatre, format, type, screen_count, capacity, hfc_total, screens (JSONB)
movies	Movie/report metadata	id (mov-...), name, release, created_at, owner_email
movie_sheets	The Univer spreadsheet blob	movie_id (PK), data (JSONB), version, updated_by, updated_at
movie_shares	Per-user movie access	(movie_id, email) PK, role (editor/viewer), created_at
reps	Representatives	id, name, email, phone, region, color, status, role_id
reports	Rep sales submissions	id, rep_id, key, theatre/screen/movie info, seats/collection totals, occupancy, status, submitted_at

Note: roles and admin-managed users currently live in browser localStorage (svf.roles.v1, svf.users.v1), not Postgres - a candidate for future migration to the DB.

7. The Excel -> Univer Rendering Pipeline (deep dive)

xlsxToUniver.ts is the heart of the Movies workspace and the focus of most recent work. It:

1. Parses the workbook with ExcelJS and reads the real theme palette.
2. Pass 1 - captures every cell with a value or style; tracks the first bordered row (the table top) to separate the heading block.
3. Pass 2 - captures colored-fill empty cells within the data range so a colored column reads continuously (the green audience columns), not only where a number sits.
4. Heading white-fill - every cell above the first bordered row gets a white fill so the black gridlines don't show through the title area.
5. Rebuilds merged ranges, column widths, and row heights (points -> pixels, floored so 14pt text isn't cramped).
6. Emits the sheet with black gridlines on, the grid clamped to the true data extent, and the special 'Spl-1/2/3' columns hidden (toggleable in the toolbar).

Deliberate design choices made during fidelity work:

- Column count trimmed to the real content extent (no wall of empty bold columns).
- Black gridlines act as the uniform 'bold grid' the reports use.
- The source Excel's own hidden-column flags are intentionally NOT preserved - those templates hide the Noon/Matinee show columns even though those carry green audience data the user must see.

8. Current Status & Open Items (as of 30 Jul 2026)

In progress - spreadsheet color fidelity:

- **Green audience columns** (C3D69B) - capacity + audience columns (E, O, Q, S, U, W, AA, AE) should read as continuous green stripes like the source.
- **Yellow gross columns** (FFFF00) - a 'continuous-green' gap-fill experiment (Phase 4) also filled the yellow collection columns solid, painting yellow into empty rows where the source has none. Investigation confirmed, against the source file:
 - The PREMIERE SHOW sheet has no yellow (green + white only).

- The day sheets have yellow only on data rows - 0 yellow on empty rows (verified on 15TH DAY: 4,014 yellow cells on data rows, 0 on empty rows).
- Conclusion: the spurious yellow is baked into movies uploaded while the gap-fill code was live; the current converter has no gap-fill and renders faithfully. Fix path: re-upload affected movies (or restrict any gap-fill strictly to the green color).

Not committed: the Phase 4 investigation and edits are local only.

9. Development & Deployment Notes

- **Env & secrets** - DATABASE_URL and AUTH_SECRET live only in the gitignored frontend/.env.local; never committed. DB connects with SSL.
- **Seeding** - npm run seed loads theatres.json into the theatres table.
- **Build config** - ESLint and TS errors are skipped during next build so Vercel deploys don't block on lint.
- **Deployment** - Vercel auto-deploys from GitHub subha-sh-11/SVF; production can lag local until a push/deploy happens.
- **Type-checking safely** - verify with node_modules/.bin/tsc --noEmit rather than running next build while the dev server is up.
- **Default credentials** (dev, env-overridable): admin admin@svf.in / Admin@321, user user@svf.in / 123456789.

10. Commit Reference (newest -> oldest)

d442611	2026-07-29 17:46	issue fixed
4dbe7b8	2026-07-29 12:28	issue fixed
bb9a090	2026-07-27 18:06	Issues Fixed
c5f63aa	2026-07-27 17:31	Issues Fixed
15c7cb6	2026-07-27 16:52	Issues Fixed
15614fc	2026-07-27 13:53	Issues Fixed
4da7022	2026-07-24 13:36	ver2
b30fe46	2026-07-23 16:45	ver2
b6b7fa1	2026-07-23 16:32	ver2
d5fe800	2026-07-23 16:22	ver2
b911dc2	2026-07-23 11:38	Fix movie-open crash + collab blank + loading card
8f6b548	2026-07-22 18:02	Univer spreadsheets, app-user accounts, sharing, and fixes
ec92f11	2026-07-21 17:50	Skip ESLint during build so Vercel deploy passes
811c69b	2026-07-21 17:47	Skip ESLint during build so Vercel deploy passes
491d1d3	2026-07-21 17:28	Skip ESLint during build so Vercel deploy passes
02bf7d2	2026-07-21 17:20	V1
bcff6ed	2026-07-16 17:47	DB-backed theatres + admin login
434d4d5	2026-07-16 13:14	Add frontend app
5cc0127	2026-07-16 13:11	Initial commit: SVF admin frontend